

### Практическая работа №3

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке

<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Добавлено объявление новых тем в заголовочном файле

```
namespace QSourceHighlighter {  
  
class QSourceHighlighter : public QSyntaxHighlighter  
{  
public:  
    enum Themes {  
        Monokai = 1,  
        DarkTheme = 2,  
        LightTheme = 3,  
    };  
};
```

2) По аналогии с функцией monokai() добавлены две темы, их вызовы добавлены в функцию theme()

```
53 static QHash<QSourceHighlighter::Token, QTextCharFormat> dark()  
54 {  
55     QHash<QSourceHighlighter::Token, QTextCharFormat> _formats = formats();  
56  
57     _formats[QSourceHighlighter::Token::CodeBlock].setForeground(QColor(135, 133, 115));  
58     _formats[QSourceHighlighter::Token::CodeBlock].setBackground(QColor(34, 34, 27));  
59     _formats[QSourceHighlighter::Token::CodeKeyWord].setForeground(QColor(95, 145, 130));  
60     _formats[QSourceHighlighter::Token::CodeString].setForeground(QColor(125, 151, 38));  
61     _formats[QSourceHighlighter::Token::CodeComment].setForeground(QColor(95, 94, 78));  
62     _formats[QSourceHighlighter::Token::CodeType].setForeground(QColor(186, 98, 54));  
63     _formats[QSourceHighlighter::Token::CodeOther].setForeground(QColor(157, 108, 124));  
64     _formats[QSourceHighlighter::Token::CodeNumLiteral].setForeground(QColor(174, 115, 19));  
65     _formats[QSourceHighlighter::Token::CodeBuiltIn].setForeground(QColor(165, 152, 13));  
66  
67     return _formats;  
68 }  
69  
70 static QHash<QSourceHighlighter::Token, QTextCharFormat> light()  
71 {  
72     QHash<QSourceHighlighter::Token, QTextCharFormat> _formats = formats();  
73  
74     _formats[QSourceHighlighter::Token::CodeBlock].setForeground(QColor(34, 34, 27));  
75     _formats[QSourceHighlighter::Token::CodeBlock].setBackground(QColor(255, 255, 255));  
76  
77     _formats[QSourceHighlighter::Token::CodeKeyWord].setForeground(QColor(95, 145, 130));  
78     _formats[QSourceHighlighter::Token::CodeString].setForeground(QColor(125, 151, 38));  
79     _formats[QSourceHighlighter::Token::CodeComment].setForeground(QColor(95, 94, 78));  
80     _formats[QSourceHighlighter::Token::CodeType].setForeground(QColor(186, 98, 54));  
81     _formats[QSourceHighlighter::Token::CodeOther].setForeground(QColor(157, 108, 124));  
82     _formats[QSourceHighlighter::Token::CodeNumLiteral].setForeground(QColor(174, 115, 19));  
83     _formats[QSourceHighlighter::Token::CodeBuiltIn].setForeground(QColor(165, 152, 13));  
84  
85     return _formats;  
86 }  
87
```

```

QHash<QSourceHighlighter::Token, QTextCharFormat>
QSourceHighlighterTheme::theme(QSourceHighlighter::Themes theme) {
    switch (theme) {
    case QSourceHighlighter::Themes::Monokai:
        return monokai();
    case QSourceHighlighter::Themes::DarkTheme:
        return dark();
    case QSourceHighlighter::Themes::LightTheme:
        return light();
    default:
        return def();
    }
}

```

и в интерфейс программы

```

void MainWindow::initThemesComboBox()
{
    ui->themeComboBox->addItem("Default", 0);
    ui->themeComboBox->addItem("Monokai", QSourceHighlighter::Themes::Monokai);
    ui->themeComboBox->addItem("Dark Theme", QSourceHighlighter::Themes::DarkTheme);
    ui->themeComboBox->addItem("Light Theme", QSourceHighlighter::Themes::LightTheme);

    ui->themeComboBox->setCurrentIndex(0);
}

```

3)Добавлена функция смены фона при изменении темы

```

478 void MainWindow::themeChanged(int) {
479     QSourceHighlighter::Themes theme = (QSourceHighlighter::Themes)ui->themeComboBox->currentData().toInt();
480     highlighter->setTheme(theme);
481     applyEditorBackground(theme);
482 }
483
484 void MainWindow::applyEditorBackground(QSourceHighlighter::Themes theme) {
485     auto themeFormats = QSourceHighlighterTheme::theme(theme);
486     QTextCharFormat blockFormat = themeFormats.value(QSourceHighlighter::CodeBlock);
487     QColor bgColor = blockFormat.background().color();
488     // if(!bgColor.isValid()) {
489     //     bgColor = "ffffff";
490     // }
491     QString styleSheet = QString("QPlainTextEdit {background-color: %1; }").arg(bgColor.name());
492     ui->plainTextEdit->setStyleSheet(styleSheet);
493     ui->statusbar->showMessage(styleSheet);
494 }
495
496

```

4) при сохранении файла в JSON формате сохраняется тема, при открытии тема подгружается

```

        return;
    }
    QFile file(fileName);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        ui->statusbar->showMessage("Ошибка: не удалось открыть файл для записи");
        return;
    }
    QJsonObject root;
    root["language"] = ui->langComboBox->currentText();
    root["theme"] = ui->themeComboBox->currentData().toInt();
    root["text"] = ui->plainTextEdit->toPlainText();

    QJsonDocument doc(root);
    file.write(doc.toJson());

    file.close();

    isUnnamed = false;
    currentFile = fileName;
    ui->statusbar->showMessage("Файл " + fileName + " сохранен", 3000);
}

QJsonParseError error;
QJsonDocument doc = QJsonDocument::fromJson(file.readAll(), &error);
file.close();
if (error.error != QJsonParseError::NoError) {
    ui->statusbar->showMessage("Ошибка парсинга.", 3000);
    return;
}

QJsonObject obj = doc.object();
QString lang = obj["language"].toString();
int theme = obj["theme"].toInt();
QString text = obj["text"].toString();

ui->plainTextEdit->setPlainText(text);

if (!lang.isEmpty()) {
    ui->langComboBox->setCurrentText(lang);
}

```